

Go Observability & Resilience Handbook

Production Engineering Reference & Reviewer Specification

Section 1: Structured Logging & Correlation

Standardize on Go 1.21+ 'log/slog' for JSON Logging

All production microservices must use the standard library log/slog package configured with `slog.NewJSONHandler(os.Stderr, ...)`. Third-party logging wrappers or unformatted `fmt.Println` logging are strictly prohibited in application code.

Context-Aware Call Sites & Trace Correlation

Always invoke loggers using context-aware methods (e.g., `slog.InfoContext(ctx, ...)` and `slog.ErrorContext(ctx, ...)`). Handlers must extract active W3C trace and span IDs from the context and append them as top-level JSON fields (`trace_id`, `span_id`) for distributed log correlation.

Dynamic Verbosity Control with `slog.LevelVar`

Configure service loggers using an atomic `slog.LevelVar` pointer. This enables operational teams to dynamically toggle logging verbosity (e.g., switching from INFO to DEBUG) at runtime via an HTTP administrative endpoint or signal without process restarts.

PII Redaction & Allocation Hygiene

Implement `slog.LogValuer` on sensitive domain models to automatically redact passwords, API keys, and PII before serialization. On high-frequency execution paths, use strongly-typed attributes (e.g., `slog.String()`, `slog.Int64()`) rather than variadic key-value pairs to eliminate heap allocations.

Idiomatic `slog` Configuration Example:

```
var logLevel = new(slog.LevelVar) // Defaults to InfoLevel

type User struct { Password string; Token string }
func (u User) LogValue() slog.Value {
    return slog.GroupValue(slog.String("password", "[REDACTED]"))
}

logger := slog.New(slog.NewJSONHandler(os.Stderr, &slog.HandlerOptions{Level: logLevel}))
slog.SetDefault(logger)
slog.InfoContext(ctx, "user authenticated", slog.String("user_id", id))
```

Section 2: Metrics Instrumentation & The Telemetry Triad

RED & USE Metrics Frameworks

Microservices must expose standardized operational metrics using Prometheus or OpenTelemetry SDKs. Apply the **RED** method (Rate, Errors, Duration) for request-driven endpoints, and the **USE** method (Utilization, Saturation, Errors) for host/infrastructure resources.

Metric Types & Explicit Histogram Bucket Hygiene

Use **Counters** for monotonically increasing values (requests served, total errors), **Gauges** for instantaneous state (active goroutines, queue depth), and **Histograms** for latency distributions. Explicitly define custom histogram latency buckets tailored to service SLAs (e.g., `[]float64{0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5}`) to prevent high-cardinality memory explosions.

The Telemetry Triad Workflow

Leverage the three telemetry pillars in a unified diagnostic pipeline: **Metrics** detect anomalous deviations (e.g., spike in HTTP 5xx rate), **Traces** isolate the failing microservice boundary or database span, and **Correlated Logs** (linked via `trace_id`) reveal root-cause stack traces.

Section 3: Distributed Tracing & Span Hygiene

W3C Trace Context Propagation

All inbound and outbound HTTP and gRPC network transports must be wrapped with OpenTelemetry instrumentation middleware (e.g., `otelhttp`, `otelgrpc`). This ensures W3C traceparent and tracestate headers seamlessly propagate across microservice boundaries.

Manual Span Scope & Immediate Deferral

Create explicit child spans for heavy internal calculations, database transactions, or external SDK calls using `tracer.Start(ctx, "operation_name")`. Always defer `span.End()` immediately following span creation to guarantee clean span closure upon function return.

Attribute Recording vs. Micro-Span Proliferation

Record high-cardinality operational metadata (e.g., customer tier, payload byte size) as key-value attributes on existing spans using `span.SetAttributes(...)` or `span.AddEvent(...)`. Do NOT create short-lived 'micro-spans' for fast, trivial in-memory helper functions.

Idiomatic Distributed Tracing Pattern:

```
func (s *OrderService) Process(ctx context.Context, order *Order) error {
    ctx, span := tracer.Start(ctx, "OrderService.Process",
        trace.WithAttributes(attribute.String("order.id", order.ID)))
    defer span.End()

    if err := s.repo.Save(ctx, order); err != nil {
        span.RecordError(err)
        span.SetStatus(codes.Error, "failed to save order")
        return fmt.Errorf("saving order: %w", err)
    }
    return nil
}
```

Section 4: Resilience, Fault Isolation & Recovery

Exponential Backoff with Full Jitter

Client retries against transient network errors or HTTP 503 responses MUST use exponential backoff combined with randomized Full Jitter (`sleep = random_between(0, min(cap, base * 2^attempt))`). Pure deterministic retries create synchronized retry storms that crush recovering dependencies.

Circuit Breaking ('sony/gobreaker')

Isolate unreliable external downstream dependencies using stateful circuit breakers (e.g., `sony/gobreaker/v2`). Configure failure thresholds so that sustained remote outages trip the breaker into an Open state, instantly failing fast to conserve local CPU and memory resources.

Idempotency Safeguards for Mutation Retries

All non-idempotent HTTP mutation endpoints (POST, PATCH) must require an Idempotency-Key header. Persist request execution results keyed by idempotency tokens in a fast storage layer (e.g., Redis/Postgres) to safely return cached responses upon upstream retries.

Panic Recovery Middleware

Install panic recovery middleware at top-level HTTP/gRPC transport boundaries. Recovered panics must be logged as high-severity errors with complete stack traces, increments to panic counter metrics, and return a clean HTTP 500 / gRPC Internal error to prevent silent process crashes.

Section 5: Health Probes & Orchestration Boundaries

Explicit Probe Separation (/livez, /readyz, /startupz)

Services must expose three decoupled health endpoints for orchestrators like Kubernetes: /startupz (initialization completion), /livez (process execution health), and /readyz (traffic handling readiness).

Process-Local Liveness Probes

Liveness checks (/livez) MUST remain strictly local to the process (e.g., verifying memory/goroutine health or event loop responsiveness). **NEVER ping external databases, caches, or remote APIs inside liveness checks**—if a database experiences a temporary hiccup, failing liveness probes will trigger cluster-wide cascading pod reboots.

Dependency-Aware Readiness Probes

Readiness checks (/readyz) MUST verify critical dependency connectivity (e.g., db.PingContext(ctx)) and local warm-up state. When a backend dependency degrades, /readyz returns HTTP 503, signaling Kubernetes to temporarily remove the pod from service endpoints without restarting it.

Section 6: Graceful Shutdown & Load Shedding

Readiness Flip & Ingress Propagation Delay

Upon receiving a termination signal (SIGTERM/SIGINT), immediately flip the readiness probe (/readyz) to return HTTP 503. Pause execution for a brief propagation window (3–5 seconds) before closing port listeners to allow Kubernetes ingress controllers and cloud load balancers to update their active endpoint routing tables.

Deterministic 5-Step Teardown Protocol

Execute process teardown in strict sequence: (1) Flip readiness to 503 & pause, (2) Stop accepting new network requests, (3) Drain active requests using server.Shutdown(ctx) with a bounded timeout budget (e.g., 15–20s), (4) Close DB connection pools, message brokers, and caches, (5) Flush logger buffers and exit.

Active Load Shedding & Concurrency Limiting

Protect services from overload using concurrency limiters (e.g., golang.org/x/sync/semaphore). When active request concurrency exceeds maximum safe thresholds, shed load immediately by returning HTTP 429 Too Many Requests or HTTP 503 Service Unavailable to prevent out-of-memory (OOM) process crashes.

Section 7: Reviewer Quick-Reference Grading Matrix

Category & Focus Area	Review Standard & Verification	Code Smell / Anti-Pattern Indicator	Idiomatic Remediation Snippet
Structured Logging	Uses 'slog' with JSON handler and context-aware methods.	fmt.Printf or log.Println; unformatted logs; missing trace_id.	slog.InfoContext(ctx, "msg", slog.String("k", v))
PII Protection	Implements slog.LogValuer to redact sensitive fields.	Logging raw User struct exposing passwords or secrets.	func (u User) LogValue() slog.Value { ... }
Metrics Buckets	Histogram latency buckets configured with explicit boundaries.	Default buckets or unbounded dynamic metric labels.	prometheus.NewHistogram(prometheus.HistogramOpts{Buckets: [...]})
Tracing Propagation	HTTP/gRPC clients wrap transport with OpenTelemetry.	Manual HTTP requests missing traceparent headers.	client := &http.Client{Transport: otelhttp.NewTransport(...)}
Span Lifecycle	Tracer starts span and defers span.End() immediately.	Spans left unclosed or created for minor in-memory helpers.	ctx, span := tracer.Start(ctx, "op"); defer span.End()
Retry Jitter	Exponential retries combine base backoff with Full Jitter.	Fixed-interval retries (time.Sleep) causing retry storms.	sleep := rand.Float64() * math.Min(cap, base * math.Pow(2, attempt))
Panic Recovery	Top-level transport middleware catches and logs panics.	Uncaught panic in HTTP handler crashing the entire pod.	defer func() { if r := recover(); r != nil { logPanic(r) } }()
Liveness Probes	Liveness check (/livez) tests process execution state only.	Liveness check calling db.PingContext(), triggering reboot loops.	mux.HandleFunc("/GET /livez", func(w, r) { w.WriteHeader(200) })
Shutdown Propagation	SIGTERM flips /readyz to 503 and pauses before server.Shutdown.	Stopping HTTP server immediately on SIGTERM, dropping in-flight HTTP requests.	isShuttingDown.Store(true); time.Sleep(3*time.Second); server.Shutdown(ctx)
Load Shedding	Concurrency limits reject excess traffic before memory exhaustion.	Service accepts unbounded requests until OOM killed.	if !sem.TryAcquire(1) { http.Error(w, "Overloaded", 503); return }